



Protocol Audit Report

Version 1.0

Cyfrin.io

May 25, 2026

Protocol Audit Report

Cyfrin.io

March 7, 2023

Prepared by: Cyfrin Lead Auditors: - xxxxxxxx

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the password on-chain makes it visible to anyone, rendering it no longer private
 - * [H-2] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec documents a parameter that does not exist, rendering it incorrect

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
1 2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
1 ./src/  
2 #-- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsides: No one else should be able to set or read the password.

Executive Summary

Add some notes about how the audit went, types of things you found, etc.

We spent X hours with Z auditors using Y tools. etc

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone, rendering it no longer private

Description: All data stored on-chain is publicly visible and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be private and only accessed through

the `PasswordStore::getPassword` function, which is intended to be called only by the owner of the contract.

Impact: Anyone can read the private password, severely breaking the intended functionality of the protocol.

Proof of Concept:

The test case below demonstrates how anyone can read the password directly from the blockchain.

1. Create a locally running chain:

```
1 make anvil
```

- ## 2. Deploy the contract to the chain:

```
1 make deploy
```

3. Run the storage tool. We use 1 because that is the storage slot of `s_password` in the contract:

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

4. You'll get an output that looks like this:

[illegible]

5. You can then parse that hex string with:

[illegible]

6. Which will produce the following output:

```
1 myPassword
```

Recommended Mitigation: Due to this vulnerability, the overall architecture of the contract should be reconsidered. One approach would be to encrypt the password off-chain and store only the encrypted value on-chain. This would require the user to remember a separate off-chain key to decrypt the password. Additionally, you would likely want to remove the view function, as you would not want the user to accidentally send a transaction with the plaintext password that could expose their decryption key.

[H-2] PasswordStore::setPassword has no access controls, meaning a non-owner could change the password

Informational

[I-1] The PasswordStore::getPassword natspec documents a parameter that does not exist, rendering it incorrect